CS4433 Software Engineering

Individual Project

Moderated Learning Management System

Final Report

Name: Boitumelo Rakoloi

Student number: 202321274

BSc Computer Science and Statistics (double major)

National University of Lesotho

Repository: https://github.com/boitumelorakoloi-lab/moderated-lms-202321274

Releases: v1.0 and v1.1

Due Date: 3 November 2026

Abstract: This report summarises the planning, requirements, design, implementation, testing, release and maintenance of a Moderated Learning Management System for a secondary school, in which no student sees a mark before the Head of Department (HoD) has approved it. The system is an offline C++17 console program using SQLite and Argon2id password hashing, built with a layered architecture and four design patterns. Release v1.0 meets all 19 Must and 5 Should requirements; change request CR1 (whole-class extensions) was delivered as v1.1. 104 automated tests pass, business-logic coverage is 96.4% and two static analysers report no findings. Diagrams, the traceability matrix, full test results, manuals and the risk register are in the repository and linked from each section.

1. Introduction

The school has about 800 students and four kinds of user: students, lecturers, HoDs and an administrator. Lecturers set assignments with deadlines and maximum marks and can give extensions; students hand in files and may resubmit before the deadline; late work loses marks; HoDs moderate every mark before release; and every mark change is audited. The school has old PCs and unreliable internet. I worked alone in eight phases, each with a deliverable. Three ideas run through the project: **traceability** (every requirement ID appears in the use cases, design, code and test names), **automation** (build, tests, analysis and packaging run on every push) and **change control** (design gaps and CR1 were recorded and analysed, not quietly coded).

2. Project Management

I used the **waterfall** model, required by the brief and suited to fixed requirements and a sole developer who needs clear gates, with **V-model** testing (each phase paired with a test level). Its weakness, costly late change, was controlled by a written change-request rule. Agile was rejected because the brief requires gated documents and a one-person team gains little from its ceremonies, but I used short branches and CI inside the phases. The **feasibility study** found the project technically, operationally, economically and legally feasible (free tools, old PCs reused, learners' data protected by role), on condition that the SQLite database stays on one PC, and the schedule tight. The work breakdown structure has 22 tasks; the Gantt chart showed a 35-day critical path with no float. When the hand-in moved from 7 to 2 November, I applied the planned response to risk R1: keep every Must and Should requirement, drop the two Could requirements (FR-23 notifications, FR-24 CSV export) and overlap deployment, CR1 and reporting. The highest risks, falling behind (R1) and "database is locked" with many users (R2), both happened and were handled; see the updated risk register.

3. Requirements Engineering

From the five interview statements I identified nine stakeholder groups, including parents and the Data Protection Commission, and placed them on a power-interest grid. Conflicts, ambiguities and omissions were resolved as decisions C1-C8, with the principal (sponsor) winning conflicts while the other side's need was met another way. For example, lecturers wanted immediate release but the principal wanted HoD approval first (C1): release waits for approval, but students see "Marked, awaiting moderation". The ambiguous "lose 10% per day" (C2) became 10% of the **maximum** mark per **started 24 hours**, never below zero, and zero after 72 hours (C3). The SRS (IEEE 830 structure) has 26 functional requirements, 11 measurable non-functional requirements (e.g. NFR-01: login ≤ 2 s with 800 students and 20,000 submissions) and business rules BR-01 to BR-05, prioritised with MoSCoW and traced to their source. The use case model has 21 use cases, using generalisation (an HoD is a kind of lecturer), «include» for the late penalty and audit entry and «extend» for adjusting or returning a mark. In hindsight FR-08 ("one student") was too narrow; asking about whole-class extensions during elicitation could have avoided CR1.

4. Software Design

The component diagram shows a **layered architecture**: presentation (menus), services (permissions and business rules), domain (entities and the penalty rule) and data access (the only SQL). It keeps the rules testable without a screen or real database and lets a GUI replace the console later. MVC was rejected (little benefit for a one-screen-at-a-time console, rules drift into controllers) and client-server was rejected (needs a reliable network, and SQLite must not be shared over a network folder). The class diagram has an abstract `User` with four role subclasses (`HeadOfDepartment` extends `Lecturer`) and four patterns: **Factory** (`UserFactory` turns a stored role into the right subclass), **Strategy** (`ILatePenaltyPolicy`; the rule most likely to change is replaceable, and the final mark is computed, never stored), **Repository** (four interfaces so services never see SQL and tests use an in-memory database) and **Template Method** (`RoleMenu::run` holds the shared menu loop). A Singleton connection was rejected because tests need fresh databases. Behaviour is specified by the state machine of a submission (Submitted, Late, Marked, Returned, Moderated, Released), an activity diagram and sequence diagrams for submitting, moderating and releasing. The database (schema) is in third normal form; CHECK constraints and triggers enforce mark limits and make the audit table append-only, so the database protects the rules even if the code is bypassed.

5. Implementation

The code follows the design: one folder per layer and class names taken from the class diagram (about 3,800 lines, 318 functions, average cyclomatic complexity 2.4), under a 30-rule coding standard based on the C++ Core Guidelines. One function works out the effective deadline and one strategy object the penalty, so every screen agrees; time is passed in as a parameter so tests can check deadlines to the second. **Security:** passwords are hashed with **Argon2id** (libsodium `crypto_pwhash_str`, random salt, memory-hard); salted SHA-256 is too fast to resist guessing, bcrypt truncates at 72 bytes and PBKDF2 is not memory-hard. Accounts lock for 15 minutes after five wrong passwords, login errors do not reveal which usernames exist, and permissions are checked in the services, not only the menus. **Persistence:** RAII classes `Database`, `Statement` and `Transaction` wrap SQLite; every query uses bound parameters (no SQL injection); a mark, its state change and its audit entry are one transaction; WAL mode with a busy timeout supports 20 users. Thirteen places where the submitted class diagram gave a requirement nowhere to live were recorded with their requirement rather than changed silently, and methods no requirement needed were removed.

6. Verification and Validation

All tests are automated (GoogleTest and CTest) and run on every push. Cases were designed with **equivalence partitioning** and **boundary value analysis**: the deadline is tested at −1 s, 0, +1 s, 24 h, 24 h + 1 s, 72 h and 72 h + 1 s, and likewise file size, mark range and username length.

Level (V-model)	What is tested	Tests	Result
Unit	Penalty rule, state machine, lock-out, permissions	UT-01 to UT-19 (26 cases)	Pass
Integration	Services with real SQLite repositories (in-memory)	IT-01 to IT-88 (63)	Pass
System	Real program driven by scripted keystrokes, one per use-case scenario	ST-01 to ST-13	Pass
Acceptance	NFR-01 speed and NFR-02 20 users measured; NFR-03 to NFR-11 by inspection	AT-01 to AT-10	Pass except NFR-06

104 / 104 tests pass and business-logic line coverage is **96.4%** (target 70%, enforced by CI); deliberately broken code (mutations) was caught. Login takes about 0.15 s and the largest list 0.04 s with full data. Nine defects were fixed; the serious one, **D-09**, made about 2% of 20-user CI runs fail with "database is locked" because the busy timeout was set after switching to WAL mode. It never appeared on my laptop, which is the strongest argument for automated acceptance tests in CI. NFR-06 (usability) needs real users: the test sheet is ready but has not yet been run, so it is reported as not validated. Details: full test results and defect log, traceability matrix.

7. Configuration Management and Quality Assurance

Everything is in Git except learners' data (`.gitignore` excludes database files). I used a **feature-branch workflow**: `main` is protected and always passes; each change is on a short branch merged by pull request (#1-#4 the layers, #5 CI, #7 the D-09 fix, #8 static analysis, #9 release v1.0, #10 CR1). Releases are annotated tags following Semantic Versioning. The CI pipeline has four jobs on every push: build and all tests, static analysis, coverage (fails below 70%) and packaging (publishes a GitHub release for version tags). **cppcheck** and **clang-tidy** first reported 19 findings (mainly unchecked optional values); all were fixed and cppcheck now fails the build on any new finding. Formatting is checked against `.clang-format` (88 long lines fixed). Metrics from lizard (report) guided the checklist-based self code review: `MarkingService::mark` went from complexity 15 to 5, loops became standard algorithms and a RAII class restores hidden password typing after errors.

8. Deployment

The deployment diagram follows from earlier decisions: everything runs on one **LMS PC** (Ubuntu 24.04) holding the program, the database, submitted files and backups. Users work at its keyboard or from lab PCs over **SSH on the local network**, so no internet is needed (NFR-03); each session runs its own process on the same local database file, the case WAL mode is designed for. A web application was rejected (needs a server and more security work), and Windows remote desktop allows only one user. CI builds two packages: an Ubuntu **.deb** (installs the libraries with apt) and a Windows zip with a single statically linked `lms.exe`; all system tests also passed on the Windows build, and third-party downloads are checked against SHA-256 hashes. The installation guide is written for a technician: every command, a dedicated `lms` account owning the data folder (permissions 700), SSH restricted to the program, daily backups of the database **and** the submissions folder, upgrade steps and troubleshooting. The user manual has a part per role, and the release notes list features and known limitations. Release v1.0 is tagged on the merge of pull request #9.

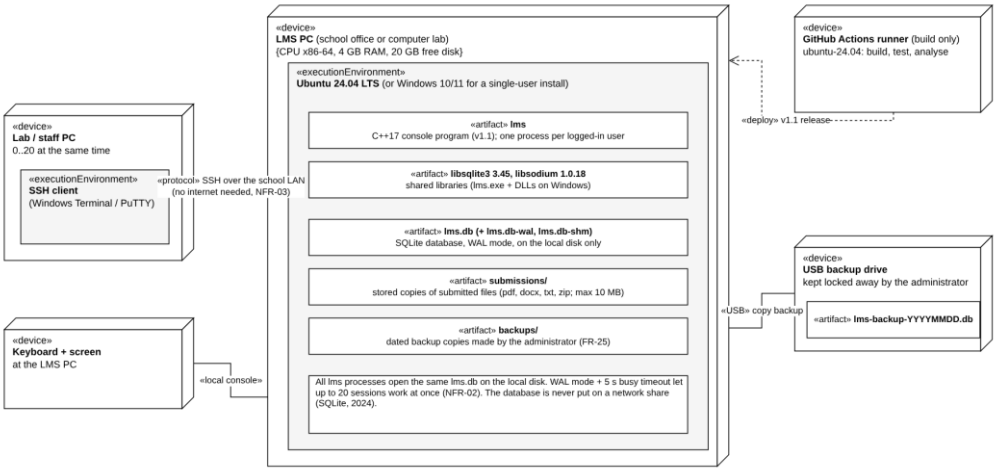


Figure 1: UML deployment diagram (source: docs/uml/9-deployment.drawio)

9. Maintenance: Change Request CR1

CR1: "A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list." It is **perfective** maintenance (Lientz and Swanson, 1980; ISO/IEC/IEEE 14764): new functionality requested by users. It is not corrective, because v1.0 met FR-08 as agreed, nor adaptive, as the environment did not change; hence a minor release, v1.1. **Impact analysis** started from FR-08 and BR-03 in the traceability matrix and a code search for every place a deadline is worked out:

Artefact	Affected
Requirements	FR-27 and BR-06 new; FR-09 changed (list shows extension type and reason); FR-08/BR-03 combined with BR-06; FR-10, FR-12, FR-15, FR-17 use the new effective deadline
Use cases / diagrams	UC-07 new flow A2; UC-16, UC-17, UC-08, UC-11 display it. Class diagram, ERD, state machine updated (changes shaded)
Classes	`Extension` (empty studentId = whole class), `AssignmentService`, `ICourseRepository`, `SqliteCourseRepository`, `Database`, `applicableExtension`, Submission/Marking/Moderation services, three menus
Tables / tests	New `class_extension` table; `submission` status update. New UT-19, IT-80 to IT-88, ST-13; IT-26 and ST-06 updated

Design: copying a row per student into the extension table was rejected (students enrolled later would miss it, and it cannot be changed in one step), as was changing the assignment's deadline (loses the original and the reason). A new table stores one class extension per assignment. **BR-06:** only the course lecturer, with a reason; the deadline must be after the normal one and any earlier class extension (it only moves later); refused once marking starts, so approved marks cannot change; the later of personal and class extension counts; covered late work moves Late → Submitted in the same transaction. A v1.0 database is upgraded automatically by adding the table. The change was merged as pull request #10 and tagged **v1.1**, with the CR1 record. **Regression:** all 93 v1.0 tests still pass and 11 new tests were added (104 / 104); the two edited tests were recorded because an edited regression test can hide a regression.

10. Professional, Legal and Ethical Issues

Data protection. Learners' names, work, marks and extension reasons are personal data, mostly of children, under Lesotho's **Data Protection Act, 2011**, which requires lawful, purpose-limited processing, appropriate security safeguards (s.20), notification of the Commission and data subjects after unauthorised access (s.23), and treats children's data as sensitive, generally needing parental consent (s.36). The design collects only what assessment needs (no addresses, birth dates or photos), enforces role-based access in the services (a student sees only their own records, tested by ST-09), keeps data on the school PC rather than the internet or cloud, and keeps it out of Git. Accounts are deactivated rather than deleted to keep records complete. Consent, retention periods and breach reporting are the school's duties and the installation guide says so. A weakness is free-text extension reasons, where a lecturer could type health details; fixed categories would be safer.

Security of assessment records. Beyond Argon2id hashing and lock-out, integrity comes from reasons for every mark change, an append-only audit log enforced by database triggers, a state machine that forbids changes after approval, and all-or-nothing release. Someone with full control of the PC could still copy the file or change the clock that decides lateness; the separate `lms` account, restricted SSH

and locked backups reduce this, and encryption and a trusted time source are future work. Stating these limits honestly is part of professional responsibility (ACM/IEEE-CS, 1999).

Accessibility. Lesotho has ratified the UN Convention on the Rights of Persons with Disabilities. Plain console text works with screen readers, everything is keyboard-only, no meaning depends on colour (errors start with "ERROR:", NFR-10), and menus are short and consistent. Typing file paths is hard for users with motor impairments, and the program has not been tested with a screen-reader user. Extensions act as reasonable adjustments, now also for a whole class.

Fairness of the late penalty. The rule is transparent, published, and applied identically by one piece of code, and basing it on the maximum mark treats weak and strong students equally; late work is always accepted with feedback. It is harsh at the edges: one minute late costs as much as 23 hours, and lateness depends on access to one PC. Extensions with recorded reasons and HoD adjustment soften this, and because the rule is a Strategy class the school could adopt a one-hour grace period, which I recommend, by adding one class.

11. Conclusion

The system meets every Must and Should requirement plus CR1, and the central rule (no mark before HoD approval) is enforced by the state machine, the services and tests at three levels. IDs made the CR1 impact analysis fast; CI from day one found D-09; the Strategy and Repository patterns made the rules testable. I would draw sequence diagrams for every Must use case (to find design gaps at the design gate), ask stakeholders about likely changes earlier, and run the usability test with real users, including a screen-reader user, much sooner. Future work: a grace period, fixed extension reasons, the two Could requirements, encryption at rest and a web front end on the existing service layer.

12. References

ACM/IEEE-CS (1999) Software Engineering Code of Ethics and Professional Practice. https://www.acm.org/code-of-ethics/software-engineering-code

Boehm, B.W. (1991) 'Software risk management: principles and practices', IEEE Software, 8(1), pp. 32–41.

DLA Piper (n.d.) Data protection laws of the world: Lesotho. https://www.dlapiperdataprotection.com/index.html?c=LS&t=law

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns. Reading, MA: Addison-Wesley.

IEEE (1998) Std 830-1998: Recommended Practice for Software Requirements Specifications. New York: IEEE.

ISO/IEC/IEEE (2022) 14764:2022 Software engineering — Maintenance. Geneva: ISO.

Lesotho (2012) Data Protection Act, 2011 (Act 5 of 2012). https://lesotholii.org/akn/ls/act/2012/5

Lientz, B.P. and Swanson, E.B. (1980) Software Maintenance Management. Reading, MA: Addison-Wesley.

McCabe, T.J. (1976) 'A complexity measure', IEEE Transactions on Software Engineering, SE-2(4), pp. 308–320.

OWASP (2024) Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Sommerville, I. (2016) Software Engineering. 10th edn. Boston: Pearson.

SQLite (2024) Appropriate Uses For SQLite. https://www.sqlite.org/whentouse.html

United Nations (2006) Convention on the Rights of Persons with Disabilities. New York: UN.

Appendices (in the repository)

Appendix	Link
A. Requirements traceability matrix (FR and NFR)	docs/appendices/A_traceability_matrix.md
B. Full test results, regression run and defect log	docs/appendices/B_full_test_results.md
C. User manual (each role)	docs/USER_MANUAL.md
D. Installation guide	docs/INSTALL.md
E. Updated risk register	docs/appendices/E_risk_register.md
F. Usability test sheet (NFR-06)	docs/appendices/F_usability_test_sheet_NFR06.pdf
UML and planning diagrams (draw.io sources and PNGs)	docs/uml/ · docs/planning/